

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3801 – Web Technologies Lab	AY: 2026–27

Assignment - 11

Full-Stack REST API Blog Management Application

1 Aim:

To develop a full-stack Blog/Post Management Application using React, Express.js, Node.js, and MongoDB Atlas. The application demonstrates REST API operations such as creating, viewing, updating, and deleting blog posts.

2 Technologies Used:

- Frontend: React.js
- Backend: Node.js and Express.js
- Database: MongoDB Atlas
- Database Driver: MongoDB Node.js Driver
- API Communication: REST API
- API Calls: JavaScript `fetch()`
- Configuration: `.env`

3 Project Structure:

The project is divided into two main parts: the React frontend and the Express.js backend.

```
Blog-App/
|
|-- app/
|   |-- src/
|       |-- components/
|           |-- PostSummary.jsx
|           |
|       |-- pages/
|           |-- Home.jsx
|           |-- Create.jsx
|           |-- Post.jsx
|           |-- Archive.jsx
|           |
|       |-- App.jsx
|       |-- index.css
|       |-- main.jsx
|
'-- server/
```

```

|-- db/
|   '-- conn.mjs
|
|-- routes/
|   '-- posts.mjs
|
|-- .env
|-- index.mjs
'-- loadEnvironment.mjs

```

4 Design:

The application follows a client-server architecture.

- React is used to build the frontend interface.
- Express.js is used to create REST API endpoints.
- MongoDB Atlas is used to store blog post data.
- The MongoDB Node.js driver is used to communicate with MongoDB.
- The React frontend communicates with the backend using HTTP requests.

The basic flow of the application is:

```

React Frontend
  |
  | HTTP Request
  v
Express.js REST API
  |
  | MongoDB Driver
  v
MongoDB Atlas
  |
  | JSON Response
  v
React Frontend

```

5 REST API Endpoints:

The application provides the following REST API operations:

Method	Endpoint	Operation
GET	/api/posts	View all posts
GET	/api/posts/:id	View a single post
POST	/api/posts	Create a new post
PUT	/api/posts/:id	Update an existing post
DELETE	/api/posts/:id	Delete a post

6 Algorithm:

1. Start the Express.js server.
2. Load environment variables from the `.env` file.
3. Connect the Node.js application to MongoDB Atlas.
4. Create REST API routes for blog posts.
5. Fetch all posts from MongoDB when the Home page is opened.
6. Display each post using the `PostSummary` component.
7. Allow the user to create a new post using the Create page.
8. Send the new post to the backend using a POST request.
9. Store the post in MongoDB.
10. Allow the user to view an individual post.
11. Allow the user to edit the post using a PUT request.
12. Allow the user to delete the post using a DELETE request.
13. Update the frontend according to the API response.
14. End.

7 Code Implementation:

7.1 App.jsx

The App.jsx file manages navigation between the Home, Create, Post, and Archive pages using React state.

```
app > src > App.jsx > default
4 import Create from "../pages/Create";
5 import Post from "../pages/Post";
6 import Archive from "../pages/Archive";
7
8 function App() {
9   const [page, setPage] = useState("home");
10  const [selectedPost, setSelectedPost] = useState(null);
11
12  const renderPage = () => {
13    if (page === "home") {
14      return (
15        <Home
16          setPage={setPage}
17          setSelectedPost={setSelectedPost}
18        />
19      );
20    }
21
22    if (page === "create") {
23      return <Create setPage={setPage} />;
24    }
25
26    if (page === "post") {
27      return (
28        <Post
29          postId={selectedPost}
30          setPage={setPage}
31        />
32      );
33    }
34
35    if (page === "archive") {
36      return <Archive />;
37    }
38  };
39
40  return (
41    <div className="app">
42      <nav>
43        <button onClick={() => setPage("home")}>
44          Home
45        </button>
46
47        <button onClick={() => setPage("create")}>
48          Create Post
49        </button>
50
51        <button onClick={() => setPage("archive")}>
52          Archive
53        </button>
54      </nav>
55
56      <main>
57        {renderPage()}
58      </main>
59    </div>
60  );
61 }
62
63 export default App;
```

Figure 1: App.jsx

7.2 PostSummary.jsx

The `PostSummary.jsx` component displays a short summary of each blog post and provides a button to view the complete post.

```
app > src > components > PostSummary.jsx > default
1  function PostSummary({ post, onSelect }) {
2      return (
3          <div className="post-card">
4              <h2>{post.title}</h2>
5
6              <p>
7                  {post.content.length > 100
8                      ? post.content.substring(0, 100) + "..."
9                      : post.content}
10             </p>
11
12             <small>
13                 By {post.author}
14             </small>
15
16             <br />
17
18             <button onClick={() => onSelect(post._id)}>
19                 View Post
20             </button>
21         </div>
22     );
23 }
24
25 export default PostSummary;
```

Figure 2: PostSummary.jsx

7.3 Home.jsx

The Home page retrieves all posts from the REST API using the `fetch()` function and displays them using the `PostSummary` component.

```
app > src > pages > Home.jsx > default
1  import { useEffect, useState } from "react";
2  import PostSummary from "../components/PostSummary";
3
4  function Home({ setPage, setSelectedPost }) {
5    const [posts, setPosts] = useState([]);
6
7    useEffect(() => {
8      fetch("http://localhost:5000/api/posts")
9        .then((response) => response.json())
10       .then((data) => setPosts(data))
11       .catch((error) =>
12         console.error("Error fetching posts:", error)
13       );
14    }, []);
15
16    const selectPost = (id) => {
17      setSelectedPost(id);
18      setPage("post");
19    };
20
21    return (
22      <div>
23        <h1>Blog Posts</h1>
24
25        {posts.length === 0 ? (
26          <p>No posts available.</p>
27        ) : (
28          posts.map((post) => (
29            <PostSummary
30              key={post._id}
31              post={post}
32              onSelect={selectPost}
33            />
34          ))
35        )}
36      </div>
37    );
38  }
39
40  export default Home;
```

Figure 3: Home.jsx

7.4 Create.jsx

The Create page provides a form for entering the title, author, and content of a new blog post. The data is sent to the backend using a POST request.

```
app > src > pages > Create.jsx > default
1  import { useState } from "react";
2
3  function Create({ setPage }) {
4      const [title, setTitle] = useState("");
5      const [content, setContent] = useState("");
6      const [author, setAuthor] = useState("");
7
8      const createPost = async (e) => {
9          e.preventDefault();
10
11          if (!title || !content || !author) {
12              alert("Please fill all fields.");
13              return;
14          }
15
16          const response = await fetch(
17              "http://localhost:5000/api/posts",
18              {
19                  method: "POST",
20                  headers: {
21                      "Content-Type": "application/json"
22                  },
23                  body: JSON.stringify({
24                      title,
25                      content,
26                      author
27                  })
28              }
29          );
30
31          if (response.ok) {
32              alert("Post created successfully!");
33
34              setTitle("");
35              setContent("");
36              setAuthor("");
37
38              setPage("home");
39          }
40      };
41
42      return (
```

Figure 4: Create.jsx

```

app > src > pages > Create.jsx > default
3  function Create({ setPage }) {
41
42      return (
43          <div>
44              <h1>Create New Post</h1>
45
46              <form onSubmit={createPost}>
47                  <input
48                      type="text"
49                      placeholder="Post Title"
50                      value={title}
51                      onChange={(e) =>
52                          setTitle(e.target.value)
53                      }
54                  />
55
56                  <br /><br />
57
58                  <input
59                      type="text"
60                      placeholder="Author"
61                      value={author}
62                      onChange={(e) =>
63                          setAuthor(e.target.value)
64                      }
65                  />
66
67                  <br /><br />
68
69                  <textarea
70                      placeholder="Write your post..."
71                      value={content}
72                      onChange={(e) =>
73                          setContent(e.target.value)
74                      }
75                  />
76
77                  <br /><br />
78
79                  <button type="submit">
80                      Create Post
81                  </button>
82              </form>
83          </div>
84      );
85  }
86
87  export default Create;

```

Figure 5: Post.jsx

7.5 Post.jsx

The Post page displays an individual blog post and provides functionality for editing and deleting the post.

```
app > src > pages > Post.jsx > default
1  import { useEffect, useState } from "react";
2
3  function Post({ postId, setPage }) {
4    const [post, setPost] = useState(null);
5
6    const [editing, setEditing] = useState(false);
7
8    const [title, setTitle] = useState("");
9    const [content, setContent] = useState("");
10   const [author, setAuthor] = useState("");
11
12   const fetchPost = async () => {
13     const response = await fetch(
14       `http://localhost:5000/api/posts/${postId}`
15     );
16
17     const data = await response.json();
18
19     setPost(data);
20     setTitle(data.title);
21     setContent(data.content);
22     setAuthor(data.author);
23   };
24
25   useEffect(() => {
26     fetchPost();
27   }, [postId]);
28
29   const updatePost = async () => {
30     const response = await fetch(
31       `http://localhost:5000/api/posts/${postId}`,
32       {
33         method: "PUT",
34         headers: {
35           "Content-Type": "application/json"
36         },
37         body: JSON.stringify({
38           title,
39           content,
40           author
41         })
42       }
43     );
44
45     const data = await response.json();
46
47     setPost(data);
48     setEditing(false);
49
50     alert("Post updated successfully!");
51   };
52
53   const deletePost = async () => {
54     const response = await fetch(
55       `http://localhost:5000/api/posts/${postId}`,
56       {
57         method: "DELETE"
58       }
59     );
60
61     if (response.ok) {
62       alert("Post deleted successfully!");
63       setPage("home");
64     }
65   };
66 }
```

Figure 6: Archive.jsx

app > src > pages > Post.jsx > default

```
3   function Post({ postId, setPage }) {
65  };
66
67  if (!post) {
68    return <p>Loading...</p>;
69  }
70
71  return (
72    <div>
73      {editing ? (
74        <>
75          <h1>Edit Post</h1>
76
77          <input
78            value={title}
79            onChange={(e) =>
80              setTitle(e.target.value)
81            }
82          />
83
84          <br /><br />
85
86          <input
87            value={author}
88            onChange={(e) =>
89              setAuthor(e.target.value)
90            }
91          />
92
93          <br /><br />
94
95          <textarea
96            value={content}
97            onChange={(e) =>
98              setContent(e.target.value)
99            }
100         />
101
102         <br /><br />
103
104         <button onClick={updatePost}>
105           Save Changes
106         </button>
107
108         <button
109           onClick={() => setEditing(false)}
110         >
111           Cancel
112         </button>
113       </>
114     ) : (
115       <>
116         <h1>{post.title}</h1>
117
118         <p>
119           <strong>Author:</strong>{" "}
120           {post.author}
```

7.6 Archive.jsx

The Archive page is included as part of the application structure and provides a separate page for archived posts.

```
app > src > pages > Archive.jsx > default
1  function Archive() {
2      return (
3          <div>
4              <h1>Archive</h1>
5              <p>
6                  Archived posts will be displayed here.
7              </p>
8          </div>
9      );
10 }
11
12 export default Archive;
```

Figure 8: posts.mjs

7.7 index.mjs

The index.mjs file initializes the Express server, enables CORS and JSON parsing, connects the application to MongoDB, and registers the REST API routes.

```
server > JS index.mjs > ...
1  import express from "express";
2  import cors from "cors";
3  import { loadEnvironment } from "../loadEnvironment.mjs";
4  import { connectToDatabase } from "../db/conn.mjs";
5  import postsRouter from "../routes/posts.mjs";
6
7
8  const app = express();
9
10 const PORT = process.env.PORT || 5000;
11
12 app.use(cors());
13 app.use(express.json());
14 app.use(express.json());
15 app.use("/api/posts", postsRouter);
16
17 app.get("/", (req, res) => {
18     res.send("Blog REST API is running!");
19 });
20
21 connectToDatabase()
22     .then(() => {
23         app.listen(PORT, () => {
24             console.log(
25                 `Server running at http://localhost:${PORT}`
26             );
27         });
28     })
29     .catch((error) => {
30         console.error(error);
31     });
```

Figure 9: conn.mjs

7.8 posts.mjs

The `posts.mjs` file contains the REST API routes for creating, retrieving, updating, and deleting blog posts.

```
server > routes > JS posts.mjs > default
1  import express from "express";
2  import { ObjectId } from "mongodb";
3  import { getDatabase } from "../db/conn.mjs";
4
5  const router = express.Router();
6
7
8  // GET - View all posts
9  router.get("/", async (req, res) => {
10     try {
11         const db = getDatabase();
12
13         const posts = await db
14             .collection("posts")
15             .find({})
16             .toArray();
17
18         res.json(posts);
19     } catch (error) {
20         res.status(500).json({
21             message: error.message
22         });
23     }
24 });
25
26 // GET - View individual post
27 router.get("/:id", async (req, res) => {
28     try {
29         const db = getDatabase();
30
31         const post = await db
32             .collection("posts")
33             .findOne({
34                 _id: new ObjectId(req.params.id)
35             });
36
37         if (!post) {
38             return res.status(404).json({
39                 message: "Post not found"
40             });
41         }
42
43         res.json(post);
44     } catch (error) {
45         res.status(500).json({
46             message: error.message
47         });
48     }
49 });
50
51 // POST - Create new post
52 router.post("/", async (req, res) => {
53     try {
54         const db = getDatabase();
55
56         const post = {
57             title: req.body.title,
58             content: req.body.content,
59             author: req.body.author,
60             createdAt: new Date()
61         };
62
63         await db
64             .collection("posts")
65             .insertOne(post);
66         res.status(201).json(post);
67     } catch (error) {
68         res.status(500).json({
69             message: error.message
70         });
71     }
72 });
```

Figure 10: `loadEnvironment.mjs`

```

server > routes > JS posts.mjs > default
81 // PUT - Update post
82 router.put("/:id", async (req, res) => {
83   try {
84     const db = getDatabase();
85
86     const updatedPost = {
87       title: req.body.title,
88       content: req.body.content,
89       author: req.body.author,
90       updatedAt: new Date()
91     };
92
93     const result = await db
94       .collection("posts")
95       .findOneAndUpdate(
96         { _id: new ObjectId(req.params.id) },
97         { $set: updatedPost },
98         { returnDocument: "after" }
99       );
100
101     if (!result) {
102       return res.status(404).json({
103         message: "Post not found"
104       });
105     }
106
107     res.json(result);
108   } catch (error) {
109     res.status(500).json({
110       message: error.message
111     });
112   }
113 });
114
115 // DELETE - Delete post
116 router.delete("/:id", async (req, res) => {
117   try {
118     const db = getDatabase();
119
120     const result = await db
121       .collection("posts")
122       .deleteOne({
123         _id: new ObjectId(req.params.id)
124       });
125
126     if (result.deletedCount === 0) {
127       return res.status(404).json({
128         message: "Post not found"
129       });
130     }
131
132     res.json({
133       message: "Post deleted successfully"
134     });
135   } catch (error) {
136     res.status(500).json({
137       message: error.message
138     });
139   }
140 }
141 });
142
143
144 export default router;

```

Figure 11: Home Page

7.9 conn.mjs

The `conn.mjs` file establishes the connection between the Node.js backend and MongoDB Atlas using the MongoDB Node.js driver.

```
server > db > JS conn.mjs >  getDatabase
1  import { MongoClient } from "mongodb";
2  import "../loadEnvironment.mjs";
3
4  const client = new MongoClient(process.env.MONGODB_URI);
5
6  let db;
7
8  export async function connectToDatabase() {
9    try {
10      await client.connect();
11
12      db = client.db("blogDB");
13
14      console.log("MongoDB connected successfully!");
15
16      return db;
17    } catch (error) {
18      console.error(
19        "MongoDB connection failed:",
20        error.message
21      );
22      process.exit(1);
23    }
24  }
25
26  export function getDatabase() {
27    return db;
28  }
```

Figure 12: Create Post Page

7.10 loadEnvironment.mjs

The `loadEnvironment.mjs` file loads environment variables from the `.env` file using the `dotenv` package.

```
server > JS loadEnvironment.mjs
1  import dotenv from "dotenv";
2
3  dotenv.config();
```

Figure 13: Individual Post

8 Web Page:

8.1 Home Page:

The Home page displays all blog posts retrieved from MongoDB through the REST API.

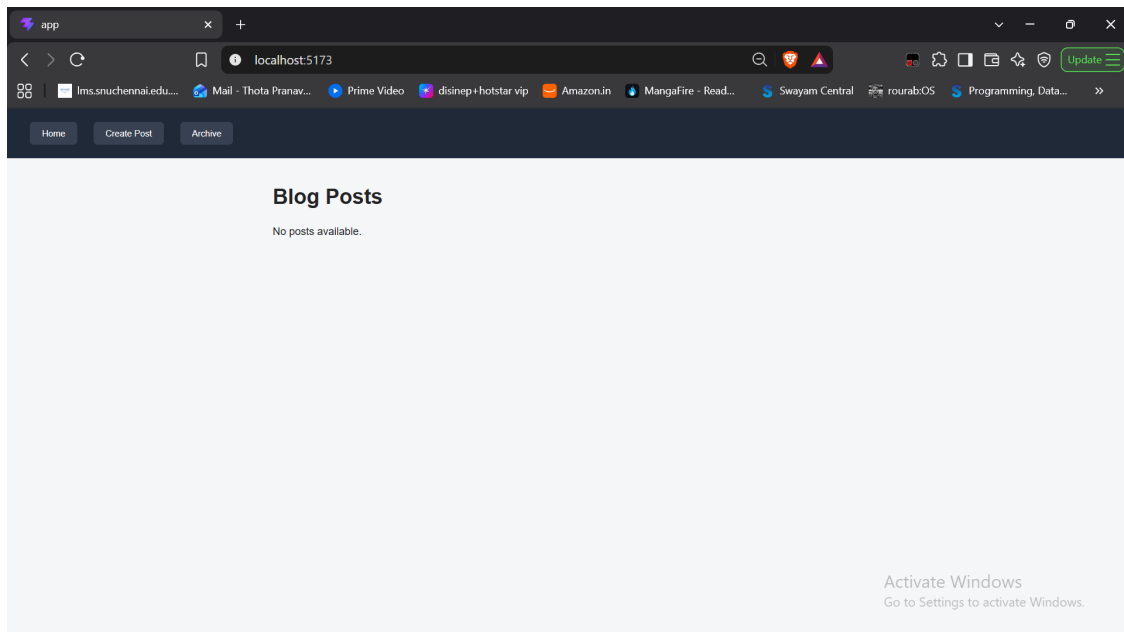
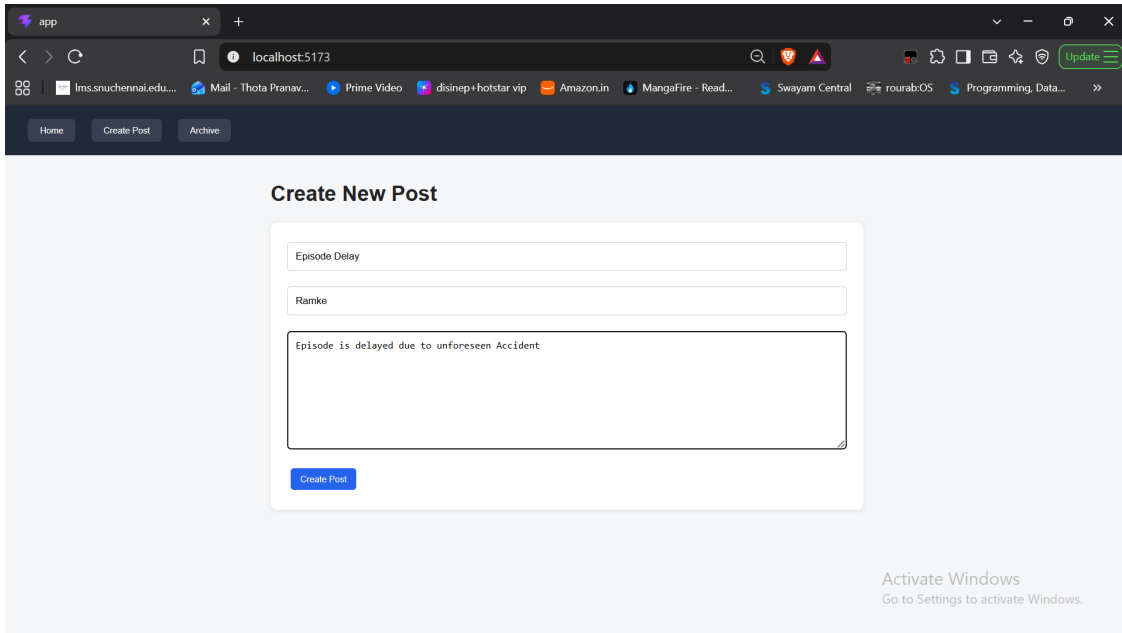


Figure 14: Edit Post

8.2 Create Post:

The Create Post page allows the user to enter the title, author, and content of a new blog post.

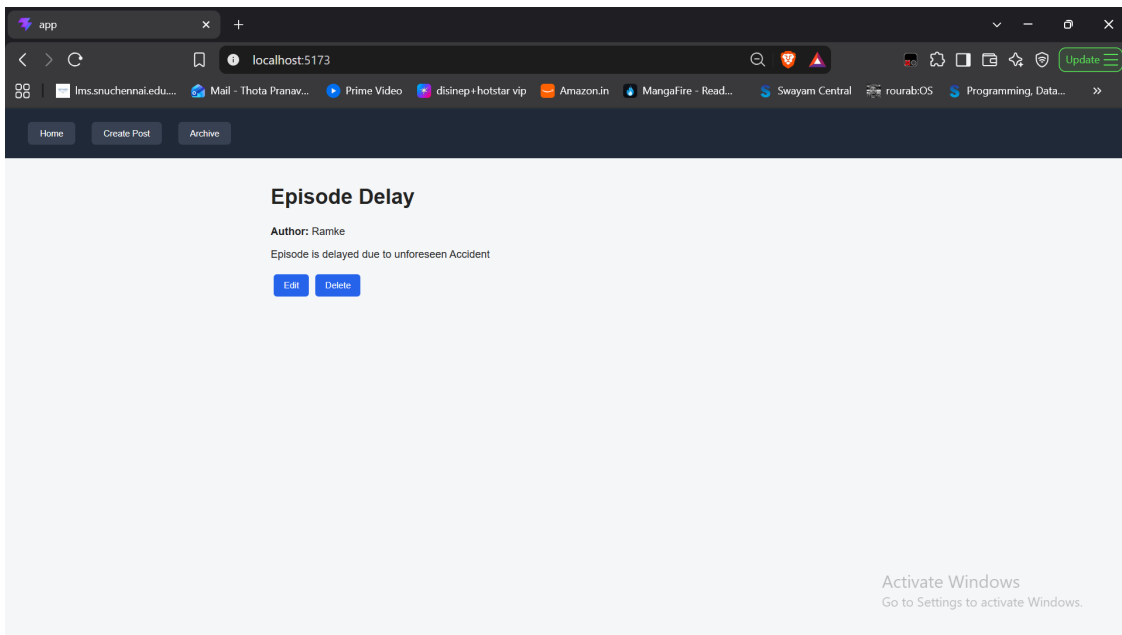


The screenshot shows a web browser window with the address bar at localhost:5173. The page has a dark navigation bar with 'Home', 'Create Post', and 'Archive' buttons. The main content area is titled 'Create New Post' and contains a form with three input fields: 'Episode Delay', 'Ramke', and a larger text area containing 'Episode is delayed due to unforeseen Accident'. A blue 'Create Post' button is at the bottom of the form. An 'Activate Windows' watermark is visible in the bottom right corner.

Figure 15: Delete Post

8.3 Individual Post:

The individual post page displays the complete details of a selected blog post.

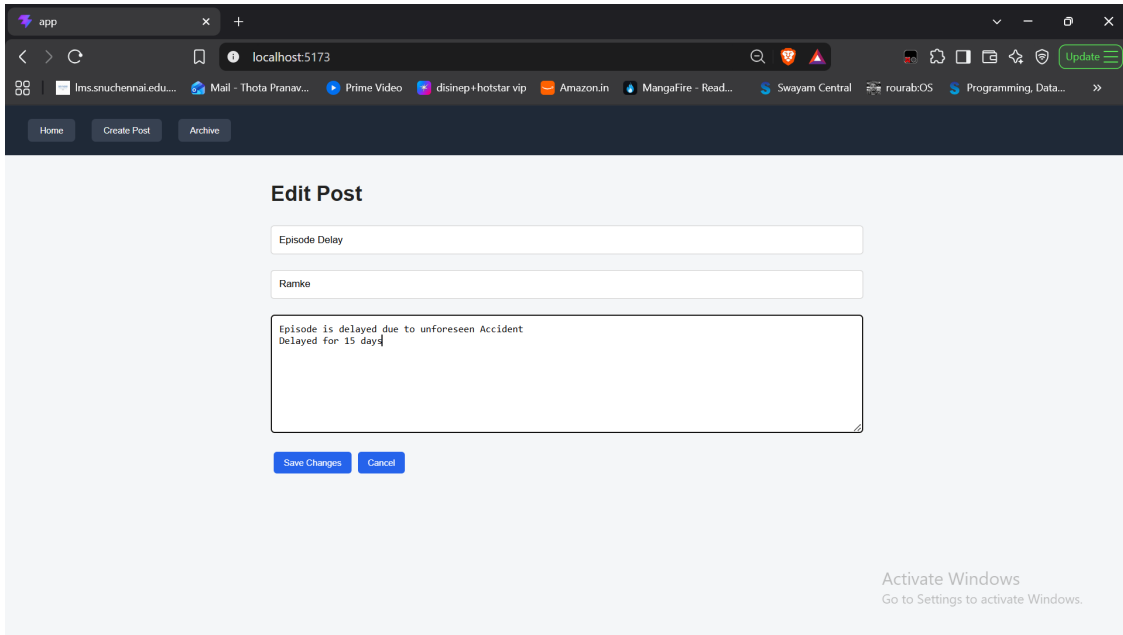


The screenshot shows the same web browser window, but the page displays the details of a specific blog post titled 'Episode Delay'. The author is listed as 'Ramke', and the content is 'Episode is delayed due to unforeseen Accident'. At the bottom of the post details, there are two blue buttons: 'Edit' and 'Delete'. The 'Activate Windows' watermark is also present in the bottom right corner.

Figure 16: GET /api/posts

8.4 Edit Post:

The Edit functionality allows the user to modify the title, author, and content of an existing post.

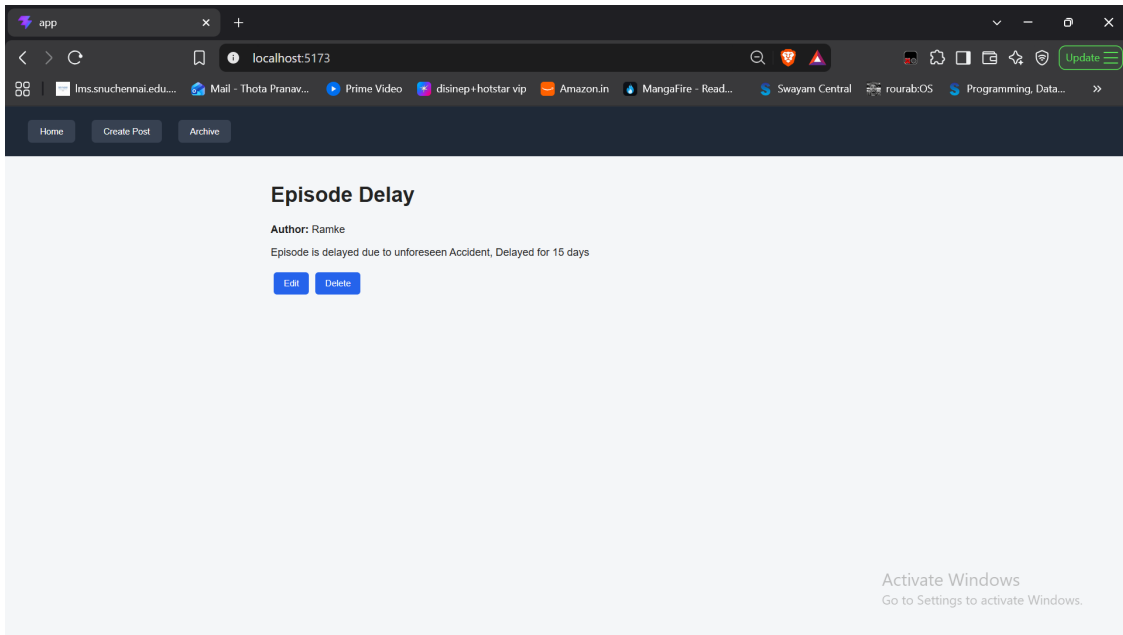


The screenshot shows a web browser window with the address bar at localhost:5173. The application has a dark header with 'Home', 'Create Post', and 'Archive' buttons. The main content area is titled 'Edit Post'. It contains three input fields: a text field for the title 'Episode Delay', a text field for the author 'Ramke', and a larger text area for the content 'Episode is delayed due to unforeseen Accident Delayed for 15 day'. Below these fields are two buttons: 'Save Changes' and 'Cancel'. At the bottom right, there is a Windows watermark: 'Activate Windows Go to Settings to activate Windows.'

Figure 17: POST /api/posts

8.5 Delete Post:

The Delete functionality removes the selected post from MongoDB and returns the user to the Home page.



The screenshot shows the same web browser window, but now displaying a specific post titled 'Episode Delay'. Below the title, it shows 'Author: Ramke' and the content 'Episode is delayed due to unforeseen Accident, Delayed for 15 days'. At the bottom of the post content, there are two buttons: 'Edit' and 'Delete'. The header and footer are identical to the previous screenshot.

Figure 18: PUT /api/posts/:id

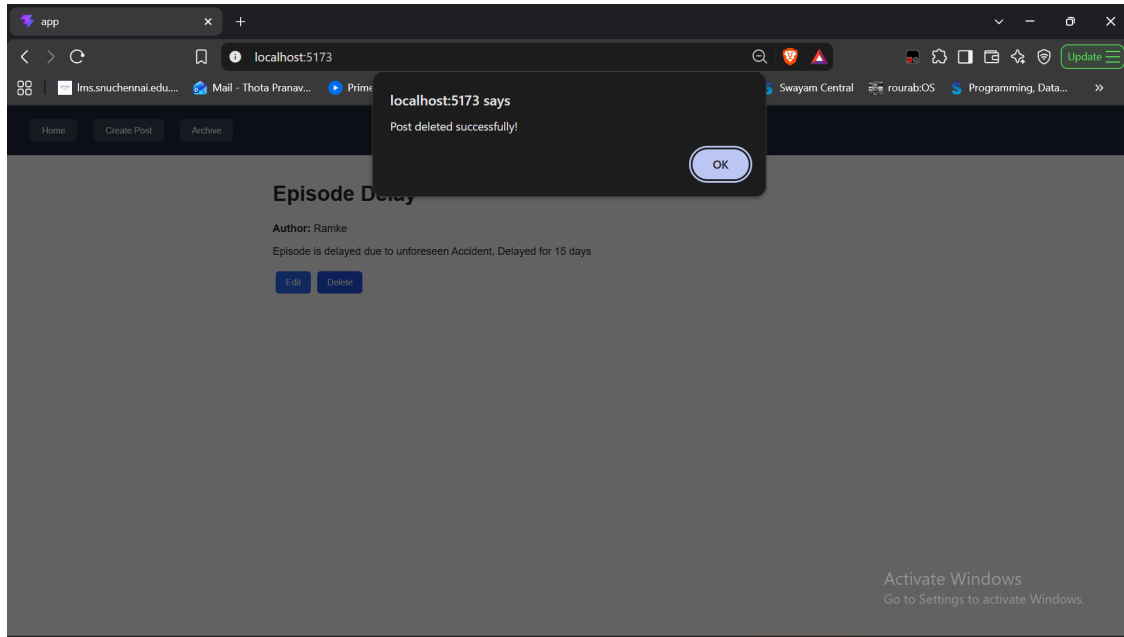


Figure 19: DELETE `/api/posts/:id`

9 Conclusion

The experiment successfully demonstrated the development of a full-stack Blog/Post Management Application using React, Express.js, Node.js, and MongoDB Atlas. The React frontend was used to create the user interface and communicate with the backend using the `fetch()` API. Express.js was used to implement RESTful API endpoints, while the MongoDB Node.js driver was used to perform database operations.

The application successfully demonstrated the major CRUD operations: creating, viewing, updating, and deleting blog posts. Thus, the experiment provided practical understanding of full-stack web development, REST APIs, React frontend development, Express.js backend development, and MongoDB database integration.

10 Code Link:

<https://github.com/PranavVarshin/Web-Technologies-Lab/Lab-11>

- NAME: T Pranav Varshin (24110413)
- Reg No: 24011101116
- Section: AIDS-B